ST3247 Midterm 2026 – Solutions

1. Estimate the integral
$$\int_{-\infty}^{\infty} e^{-3|x|} \cos(x)\, dx$$
using the Monte Carlo method.

   Remark: the exponential distribution with mean $\mu > 0$ has density $f(x) = \frac{1}{\mu} e^{-x/\mu}$ for $x > 0$ and can be generated with `np.random.exponential(scale=mu)`.
   [at least one correct decimal expected]

   Solution: $3/5 = 0.600$

```
N = 1_000_000

# Key idea: e^{-3|x|} cos(x) is an even function.
# So integral = 2 * int_0^inf cos(x) e^{-3x} dx.
# The Exp(rate=3) density is 3*e^{-3x} on [0,inf), so
# = (2/3) * E[cos(X)],  X ~ Exp(scale=1/3).

X = np.random.exponential(1/3, N)
estimate = (2/3) * np.mean(np.cos(X))
print(f"Answer: {estimate:.3f}")  # 0.600
```

2. Let $X_1, \ldots, X_{10}$ be i.i.d. $\mathrm{Uniform}(0,1)$ random variables. Estimate $\mathrm{E}\left[\max(X_1, \ldots, X_{10})\right]$.
   [at least one correct decimal expected]

   Solution: $10/11 \approx 0.909$

```
N = 1_000_000

X = np.random.uniform(0, 1, (N, 10))
estimate = np.mean(np.max(X, axis=1))
print(f"Answer: {estimate:.3f}")  # 0.909
```

3. A fair coin is tossed repeatedly. On average, how many tosses does it take until you get three consecutive heads?
[at least one correct decimal expected]

Solution: 14

```
N_sim = 500_000
total = 0
for _ in range(N_sim):
    count = 0
    streak = 0
    while streak < 3:
        count += 1
        if np.random.random() < 0.5:
            streak += 1
        else:
            streak = 0
    total += count

print(f"Answer: {total / N_sim:.1f}")  # 14.0
```

4. Estimate the integral

$$\int_{-\infty}^{\infty} \frac{\cos(x)}{1+5x^2}\,dx$$

using the Monte Carlo method.

Remark: the Cauchy distribution has density $f(x) = \frac{1}{\pi\,(1+x^2)}$ and can be generated with `np.random.standard_cauchy()`.
[at least one correct decimal expected]

Solution: Substitution $u = \sqrt{5}\,x$ gives $\int \frac{\cos(x)}{1+5x^2}\,dx = \frac{1}{\sqrt{5}} \int \frac{\cos(u/\sqrt{5})}{1+u^2}\,du = \frac{\pi}{\sqrt{5}}\,\mathrm{E}[\cos(X/\sqrt{5})]$ where $X \sim$ Cauchy. Answer: $\frac{\pi}{\sqrt{5}}\,e^{-1/\sqrt{5}} \approx 0.898$.

```
N = 1_000_000

# Key idea: 1/(1+5x^2). Sub u=sqrt(5)*x:
# = (1/sqrt(5)) int cos(u/sqrt(5))/(1+u^2) du
# = (pi/sqrt(5)) * E[cos(X/sqrt(5))], X ~ Cauchy.

X = np.random.standard_cauchy(N)
estimate = (np.pi / np.sqrt(5)) * np.mean(
    np.cos(X / np.sqrt(5)))
print(f"Answer: {estimate:.3f}")  # 0.898
```

5. Estimate the integral

$$\int_0^\infty \cos\left(\sqrt{x}\right) e^{-2x}\, dx$$

using the Monte Carlo method.

Remark: the exponential distribution with mean $\mu > 0$ has density $f(x) = \frac{1}{\mu}\exp(-x/\mu)$ and can be generated with `np.random.exponential(scale=mu)`.
[at least one correct decimal expected]

Solution: 0.385

```
N = 1_000_000

# Key idea: Exp(rate=2) has density 2*e^{-2x}.
# So e^{-2x} = (1/2) * f_{Exp(1/2)}(x).
# integral = (1/2) * E[cos(sqrt(X))], X ~ Exp(scale=1/2).

X = np.random.exponential(0.5, N)
estimate = 0.5 * np.mean(np.cos(np.sqrt(X)))
print(f"Answer: {estimate:.3f}")  # 0.385
```

6. Consider the probability distribution on $[0,3]^2$ with density $f(x,y) \propto e^{xy}$. Estimate $\mathrm{E}[X+Y]$.
[at least one correct decimal expected]

Solution: 5.21

```
N = 1_000_000

# SNIS with Uniform([0,3]^2) proposal.
# Target: f(x,y) proportional to e^{xy}
# Unnormalized weight: w = e^{x*y}

x = np.random.uniform(0, 3, N)
y = np.random.uniform(0, 3, N)
w_unnormalized = np.exp(x * y)
w_normalized = w_unnormalized / np.sum(w_unnormalized)

estimate = np.sum((x + y) * w_normalized)
print(f"Answer: {estimate:.2f}")  # 5.21
```

7. Consider the density on $\mathbb{R}$ with density $f(x) \propto (1+x^2)\, e^{-x^2}$. Estimate $\mathrm{E}[X^2]$.
[at least one correct decimal expected]

Solution: $5/6 \approx 0.833$

```
N = 1_000_000

# SNIS with Cauchy proposal (heavy tails).
# Target unnorm: p(x) = (1+x^2) * e^{-x^2}
# Cauchy unnorm:  q(x) = 1/(1+x^2)
# Unnorm weight ratio: (1+x^2)^2 * e^{-x^2}

X = np.random.standard_cauchy(N)
w_unnormalized = (1 + X**2)**2 * np.exp(-X**2)
w_normalized = w_unnormalized / np.sum(w_unnormalized)

estimate = np.sum(X**2 * w_normalized)
print(f"Answer: {estimate:.3f}")  # 0.833
```

8. Let X, Y be independent $\mathcal{N}(0,1)$ random variables. The probability $\mathrm{P}(X^3 + Y^3 > 100)$ is very small. Estimate

$$\gamma = 10^5 \times \mathrm{P}\left(X^3 + Y^3 > 100\right).$$

[one correct decimal expected]

Solution: $\gamma \approx 0.37$

```
N = 1_000_000

# IS with shifted Student-t(3) proposal centered at mu.
# Heavy tails make the estimate robust.
mu = 3.5
df = 3

# Sample from proposal
X = mu + np.random.standard_t(df, N)
Y = mu + np.random.standard_t(df, N)

# Event indicator
h = (X**3 + Y**3 > 100).astype(float)

# SNIS weights (unnormalized log-densities; constants cancel)
# Target:   N(0,1),       log p = -x^2/2
# Proposal: t(df, mu),    log q = -(df+1)/2 * log(1+(x-mu)^2/df)
log_target = -X**2/2 - Y**2/2
log_proposal = (-(df+1)/2 * np.log(1 + (X - mu)**2/df)
               + -(df+1)/2 * np.log(1 + (Y - mu)**2/df))
log_w = log_target - log_proposal
log_w -= np.max(log_w)  # numerical stability
w_unnorm = np.exp(log_w)
w_norm = w_unnorm / np.sum(w_unnorm)

gamma = 1e5 * np.sum(h * w_norm)
print(f"Answer: gamma = {gamma:.2f}")  # 0.37
```

9. Consider the Weibull distribution with shape $\alpha = 2$ and scale $\lambda = 3$, having CDF

$$F(x) = 1 - e^{-(x/3)^2}, \quad x \geq 0.$$

Using the inversion method, find the inverse CDF $\Phi = F^{-1}$ and estimate $\mathrm{E}[X]$.

Remark: the Weibull distribution with shape α and scale λ has CDF $F(x) = 1 - e^{-(x/\lambda)^\alpha}$ for $x \geq 0$, and mean $\lambda\,\Gamma(1 + 1/\alpha)$.
[at least one correct decimal expected]

Solution: From $F(x) = u$, we get $1 - e^{-(x/3)^2} = u$, so $x = 3\sqrt{-\log(1-u)}$. Thus $\Phi(u) = 3\sqrt{-\log(1-u)}$. The mean is $\mathrm{E}[X] = 3\,\Gamma(3/2) = 3\sqrt{\pi}/2 \approx 2.659$.

```
N = 1_000_000

# Inverse CDF: Phi(u) = 3 * sqrt(-log(1-u))
def Phi(u):
    return 3 * np.sqrt(-np.log(1 - u))

U = np.random.uniform(0, 1, N)
X = Phi(U)
estimate = np.mean(X)
print(f"Answer: {estimate:.3f}")  # 2.659
```

10. Let X have density $f(x) = \frac{1}{Z}\,x^2\,(2-x)^3$ on $[0, 2]$, where Z is the normalizing constant. Estimate $\mathrm{E}[X^4]$.
[at least one correct decimal expected]

Solution: $8/7 \approx 1.143$. One approach: SNIS with $\mathrm{Uniform}(0, 2)$ proposal.

```
N = 1_000_000

# SNIS with Uniform(0,2) proposal.
# Unnorm. weight: x^2 * (2-x)^3

X = np.random.uniform(0, 2, N)
w_unnormalized = X**2 * (2 - X)**3
w_normalized = w_unnormalized / np.sum(w_unnormalized)

estimate = np.sum(X**4 * w_normalized)
print(f"Answer: {estimate:.3f}")  # 1.143
```

11. Consider the probability distribution on $[0, 2]$ with density $f(x) \propto (1 + x)^2$. Estimate the standard deviation of this distribution.
[at least one correct decimal expected]

Solution: ≈ 0.509. One approach: SNIS with Uniform$(0, 2)$ proposal.

```
N = 1_000_000

# SNIS with Uniform(0,2) proposal.
# Unnormalized weight: w(x) = (1+x)^2

x = np.random.uniform(0, 2, N)
w_unnormalized = (1 + x)**2
w_normalized = w_unnormalized / np.sum(w_unnormalized)

mu = np.sum(x * w_normalized)
mu2 = np.sum(x**2 * w_normalized)
std = np.sqrt(mu2 - mu**2)
print(f"Answer: {std:.3f}")  # 0.509
```

12. Consider the distribution on $[0, \infty)$ with density $f(x) \propto e^{-\sqrt{x}}$. Estimate the variance of this distribution.
[at least one correct decimal expected]

Solution: $\mathrm{Var}(X) = 84$. Substitution $u = \sqrt{x}$ gives $\int_0^\infty x^n e^{-\sqrt{x}}\, dx = 2\int_0^\infty u^{2n+1} e^{-u}\, du = 2\,(2n+1)!$, so $\mathrm{E}[X] = 2{\cdot}3!/2{\cdot}1! = 6$, $\mathrm{E}[X^2] = 2{\cdot}5!/2{\cdot}1! = 120$, and $\mathrm{Var} = 120 - 36 = 84$.

```
N = 1_000_000

# SNIS with half-Cauchy proposal (heavy tails).
# Target unnorm:  p(x) = e^{-sqrt(x)}        on [0, inf)
# Proposal unnorm: q(x) = 1/(1+x^2)          on [0, inf)
# Sample |X| where X ~ Cauchy to get half-Cauchy.

X = np.abs(np.random.standard_cauchy(N))

log_target = -np.sqrt(X)
log_proposal = -np.log(1 + X**2)  # constants cancel in SNIS
log_w = log_target - log_proposal
log_w -= np.max(log_w)  # numerical stability
w_unnorm = np.exp(log_w)
w_norm = w_unnorm / np.sum(w_unnorm)

mu = np.sum(X * w_norm)
mu2 = np.sum(X**2 * w_norm)
var = mu2 - mu**2
print(f"Answer: Var = {var:.0f}")  # 84
```

13. Let $X_1, \ldots, X_5$ be i.i.d. Exp(1) random variables and let $S = X_1 + \cdots + X_5$. The probability $\mathrm{P}(S > 15)$ is very small. Estimate

$$\gamma = 10^4 \times \mathrm{P}\,(S > 15).$$

[at least one correct decimal expected]

Solution: $\gamma \approx 8.6$

```
N = 1_000_000

# IS: use Exp(scale=3) proposal to push samples
# toward larger values of S.
X = np.random.exponential(3, (N, 5))
S = np.sum(X, axis=1)
h = (S > 15).astype(float)

# IS weights = target / proposal
# Target:   each X_i ~ Exp(1),        log p(x_i) = -x_i
# Proposal: each X_i ~ Exp(scale=3), log q(x_i) = -log(3) - x_i/3
log_target = -S
log_proposal = -5 * np.log(3) - S / 3
log_w = log_target - log_proposal
w_unnorm = np.exp(log_w)
w_norm = w_unnorm / np.sum(w_unnorm)

gamma = 1e4 * np.sum(h * w_norm)
print(f"Answer: gamma = {gamma:.1f}")  # 8.6
```

14. A random walk $(S_0, S_1, \ldots, S_{10})$ is defined by $S_0 = 0$ and $S_t = S_{t-1} + Z_t$ where $Z_1, Z_2, \ldots$ are i.i.d. $\mathcal{N}(0,1)$. The probability that the walk stays above the line $y = t$ is very small. Estimate

$$\gamma = 10^4 \times \mathrm{P}\left(S_t \geq t \text{ for all } t = 1, 2, \ldots, 10\right).$$

[at least one correct decimal expected]

Solution: $\gamma \approx 1.50$

```
N = 1_000_000

# IS: shift each increment's mean to mu=1 so the walk
# naturally follows the barrier y = t.
# Target:   Z_i ~ N(0,1), log p(z_i) = -z_i^2/2
# Proposal: Z_i ~ N(mu,1), log q(z_i) = -(z_i-mu)^2/2
# log w = sum log p - sum log q
#       = -mu * sum(Z_i) + 10 * mu^2 / 2

mu = 1.0
Z = np.random.normal(mu, 1, (N, 10))
S = np.cumsum(Z, axis=1)
t_arr = np.arange(1, 11)
```

```
h = np.all(S >= t_arr, axis=1).astype(float)

log_w = -mu * np.sum(Z, axis=1) + 10 * mu**2 / 2
w_unnorm = np.exp(log_w)
w_norm = w_unnorm / np.sum(w_unnorm)

gamma = 1e4 * np.sum(h * w_norm)
print(f"Answer: gamma = {gamma:.2f}")  # 1.50
```

15. An importance sampling procedure with $N = 5$ particles produces the following unnormalized importance ratios:

$$r_1 = 5, \quad r_2 = 2, \quad r_3 = 1, \quad r_4 = 3, \quad r_5 = 1.$$

Compute the normalized weights and the Effective Sample Size (ESS).
[at least one correct decimal expected]

Solution: ESS $= 3.60$.

```
import numpy as np

r = np.array([5, 2, 1, 3, 1])
w = r / np.sum(r)          # normalized weights
ESS = 1 / np.sum(w**2)
print(f"Normalized weights: {w}")
print(f"ESS = {ESS:.2f}")   # 3.60
```

16. Let $p(x) \propto e^{-\sqrt{|x|}}$ be a density on $\mathbb{R}$. Estimate its normalizing constant

$$Z_p = \int_{-\infty}^{\infty} e^{-\sqrt{|x|}}\, dx.$$

Solution: $Z_p = 4$. By symmetry: $Z_p = 2\int_0^\infty e^{-\sqrt{x}}\, dx$. Substitution $u = \sqrt{x}$: $= 2\int_0^\infty 2u\, e^{-u}\, du = 4\,\Gamma(2) = 4$.

```
N = 1_000_000

# Z_p = int e^{-sqrt(|x|)} dx
# Use Cauchy proposal: q(x) = 1/(pi*(1+x^2))
# Z_p = pi * E_q[ (1+X^2) * e^{-sqrt(|X|)} ]

X = np.random.standard_cauchy(N)
g = (1 + X**2) * np.exp(-np.sqrt(np.abs(X)))
Z_p = np.pi * np.mean(g)
print(f"Answer: Z_p = {Z_p:.2f}")  # 4.00
```

17. Two points are chosen uniformly at random inside the unit square $[0,1]^2$. Estimate the expected Euclidean distance between the two points.
[at least one correct decimal expected]

Solution: 0.521

```
N = 1_000_000

x1 = np.random.uniform(0, 1, N)
y1 = np.random.uniform(0, 1, N)
x2 = np.random.uniform(0, 1, N)
y2 = np.random.uniform(0, 1, N)

d = np.sqrt((x1 - x2)**2 + (y1 - y2)**2)
estimate = np.mean(d)
print(f"Answer: {estimate:.3f}")  # 0.521
```

18. Let X, Y be independent $\mathcal{N}(0,1)$ random variables. The probability $\mathrm{P}(|X-Y|<0.01)$ is small. Use importance sampling with a coupled proposal to estimate

$$\gamma = 10^3 \times \mathrm{P}\left(|X-Y|<0.01\right).$$

Hint: to generate pairs (X,Y) that are close to each other, use the coupled proposal: $X \sim \mathcal{N}(0,1)$ and $Y = X + Z$ where $Z \sim \mathcal{N}(0,\sigma_q^2)$ with $\sigma_q = 0.01$.
[at least one correct decimal expected]

Solution: $\gamma \approx 5.64$. Under the coupled proposal, the pairs (X,Y) naturally satisfy $|X-Y|$ small, so most samples contribute to the estimate. The IS weight corrects for the mismatch between the independent target $p(x,y) = \varphi(x)\varphi(y)$ and the coupled proposal $q(x,y) = \varphi(x)\,\varphi_{\sigma_q}(y-x)$.

```
N = 1_000_000
sigma_q = 0.01

# Coupled proposal: X ~ N(0,1), Y = X + Z, Z ~ N(0, sigma_q^2).
# This generates pairs where X and Y are close.
X = np.random.normal(0, 1, N)
Z = np.random.normal(0, sigma_q, N)
Y = X + Z

# Event indicator
h = (np.abs(X - Y) < 0.01).astype(float)

# IS weights: w = target(x,y) / proposal(x,y)
# Target:   p(x,y) = phi(x) * phi(y)          [independent]
# Proposal: q(x,y) = phi(x) * phi_sigma(y-x)  [coupled]
```

```
# The phi(x) terms cancel:
# log w = log phi(y) - log phi_sigma(y-x)
log_target_y = -Y**2 / 2
log_proposal_yx = -(Y - X)**2 / (2*sigma_q**2) - np.log(sigma_q)
log_w = log_target_y - log_proposal_yx
w = np.exp(log_w)

# Standard IS: both target and proposal are normalized
# densities, so we use mean(h * w) directly.
gamma = 1e3 * np.mean(h * w)
print(f"Answer: gamma = {gamma:.2f}")  # 5.64
```

19. A random walk $(S_0, S_1, \ldots, S_{20})$ is defined by $S_0 = 0$ and $S_t = S_{t-1} + Z_t$ where $Z_1, \ldots, Z_{20}$ are i.i.d. with $\mathrm{P}(Z_t = 1) = \mathrm{P}(Z_t = -1) = 1/2$. Since $\mathrm{P}(S_{20} \geq 14)$ is very small, we use importance sampling with a biased walk where $\mathrm{P}(Z_t = 1) = 0.7$.

    Estimate the quantity
    $$\gamma = 10^4 \times \mathrm{P}\left(S_{20} \geq 14\right).$$

    Hint: if the biased walk takes n_+ steps of $+1$ and $n_- = 20 - n_+$ steps of -1, the importance weight for one path is
    $$w = \frac{(1/2)^{20}}{0.7^{n_+} \cdot 0.3^{n_-}}.$$

    [at least one correct decimal expected]

    Solution: $\gamma \approx 12.9$

```
N = 500_000
alpha = 0.7

# Simulate N biased walks of 20 steps.
# Each step: +1 with prob alpha, -1 with prob (1-alpha).
steps = (np.random.random((N, 20)) < alpha)
n_up = np.sum(steps, axis=1)
S = 2 * n_up - 20  # final position

# Event indicator
h = (S >= 14).astype(float)

# SNIS weights
# Target:   P(path) = (1/2)^20
# Proposal: P(path) = alpha^{n+} * (1-alpha)^{n-}
log_target = 20 * np.log(0.5)
log_proposal = (n_up * np.log(alpha)
               + (20 - n_up) * np.log(1 - alpha))
log_w = log_target - log_proposal
w_unnorm = np.exp(log_w)
w_norm = w_unnorm / np.sum(w_unnorm)
```

```
gamma = 1e4 * np.sum(h * w_norm)
print(f"Answer: gamma = {gamma:.1f}")  # 12.9
```

20. Let $X_1, \ldots, X_{10}$ be i.i.d. $\mathcal{N}(0,1)$ random variables. The sum $\sum_{i=1}^{10} X_i^2$ follows a $\chi^2(10)$ distribution. The probability $\mathrm{P}\left(\sum_{i=1}^{10} X_i^2 > 30\right)$ is very small. Estimate

$$\gamma = 10^5 \times \mathrm{P}\left(\sum_{i=1}^{10} X_i^2 > 30\right).$$

[at least one correct decimal expected]

Solution: $\gamma \approx 85.7$

```
N = 1_000_000

# IS: use N(0, sigma^2) proposal with sigma > 1
# to push samples toward larger values of sum(X_i^2).
sigma = np.sqrt(2)
X = np.random.normal(0, sigma, (N, 10))
S = np.sum(X**2, axis=1)
h = (S > 30).astype(float)

# IS weights = target / proposal
# Target:   N(0,1)^10,       log p = -sum(x_i^2)/2
# Proposal: N(0,sigma^2)^10, log q = -sum(x_i^2)/(2*sigma^2)
#                                    - 10*log(sigma)
log_target = -S / 2
log_proposal = -S / (2 * sigma**2) - 10 * np.log(sigma)
log_w = log_target - log_proposal
w_unnorm = np.exp(log_w)
w_norm = w_unnorm / np.sum(w_unnorm)

gamma = 1e5 * np.sum(h * w_norm)
print(f"Answer: gamma = {gamma:.1f}")  # 85.7
```